BÀI TẬP THỰC HÀNH: XÂY DỰNG AI PHÂN LOẠI ĐA DẠNG VẬT THỂ VỚI CIFAR-100

I. ĐỀ BÀI (TÌNH HUỐNG THỰC TẾ)

Bối cảnh: Bạn đang tham gia nhóm R&D của một ứng dụng thư viện ảnh thông minh (tương tự Google Photos). Ứng dụng này yêu cầu trí tuệ nhân tạo không chỉ phân biệt được "động vật" hay "xe cộ" một cách chung chung, mà phải phân loại chi tiết đến 100 danh mục khác nhau: từ hoa hồng, hoa lan, ong, bướm, cho đến xe buýt, xe tăng, v.v.

Nhiệm vụ của bạn: Xây dựng một mô hình Mạng Nơ-ron Tích chập (CNN) từ đầu (train from scratch) để phân loại bộ dữ liệu CIFAR-100.

Thông tin Bộ dữ liệu (CIFAR-100):

- Gồm 60.000 ảnh màu RGB, kích thước 32×32 pixel.
- Phân loại chi tiết thành **100 lớp** (nhãn fine).

Mục tiêu cần đạt:

Vì bài toán phân loại 100 lớp từ ảnh mờ rất khó, đôi khi AI có thể nhầm lẫn giữa "Cây phong" (Maple) và "Cây sồi" (Oak). Do đó, chúng ta sẽ đánh giá bằng 2 tiêu chí:

1. **Top-1 Accuracy (Đoán trúng phóc):** Tối thiểu **50%**.
2. **Top-5 Accuracy (Đoán trúng trong 5 lựa chọn tự tin nhất):** Tối thiểu **80%**.

II. HƯỚNG DẪN THỰC HÀNH TỪNG BƯỚC

Bước 1: Khai báo thư viện và tải dữ liệu

Gọi dữ liệu CIFAR-100 với chế độ label_mode='fine' để lấy đủ 100 lớp chi tiết.

Python
```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping
import matplotlib.pyplot as plt
import numpy as np

# Tải bộ dữ liệu CIFAR-100
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar100.load_data(label_mode='fine')
```

```
print(f"Kích thước tập huấn luyện: {x_train.shape}")
print(f"Số lượng lớp vật thể: {len(np.unique(y_train))}")
```

Bước 2: Tiền xử lý dữ liệu

Chuẩn hóa giá trị điểm ảnh (pixel) về khoảng $[0, 1]$.

Python
```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

Bước 3: Thiết kế Kiến trúc CNN Nâng cao

Để đối phó với mạng sâu và dữ liệu phân mảnh, chúng ta chèn thêm lớp BatchNormalization ngay sau lớp Tích chập (trước hàm kích hoạt ReLU). Kỹ thuật này giúp chuẩn hóa dữ liệu nội bộ, làm cho mạng hội tụ nhanh và ổn định hơn. Đồng thời, sử dụng Dropout tăng dần.

Python
```
model = models.Sequential()

# Block 1
model.add(layers.Conv2D(64, (3, 3), padding='same', input_shape=(32, 32, 3)))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.Conv2D(64, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Dropout(0.2)) # Dropout nhẹ

# Block 2
model.add(layers.Conv2D(128, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.Conv2D(128, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Dropout(0.3)) # Tăng dần Dropout

# Block 3
model.add(layers.Conv2D(256, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Dropout(0.4))

# Khối phân loại
model.add(layers.Flatten())
model.add(layers.Dense(512))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.Dropout(0.5)) # Dropout mạnh nhất ở lớp cuối
```

```
# Output layer: 100 nơ-ron tương ứng 100 lớp
model.add(layers.Dense(100, activation='softmax'))

model.summary()
```

Bước 4: Thiết lập Tăng cường dữ liệu & Callbacks

- **Data Augmentation:** Bắt buộc phải có để tạo thêm dữ liệu.
- **Early Stopping:** Một Callback cực kỳ hữu ích giúp tự động dừng huấn luyện nếu mô hình không tiến bộ sau 10 vòng lặp, đồng thời tự động khôi phục lại bộ trọng số tốt nhất.

Python

```
# Khởi tạo công cụ Tăng cường dữ liệu
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
datagen.fit(x_train)

# Tự động giảm Learning Rate nếu mô hình chững lại
lr_reduction = ReduceLROnPlateau(monitor='val_accuracy', patience=3, verbose=1, factor=0.5,
min_lr=0.00001)

# Tự động dừng sớm để chống Overfitting
early_stopping = EarlyStopping(monitor='val_accuracy', patience=10, restore_best_weights=True,
verbose=1)
```

Bước 5: Biên dịch và Huấn luyện

Trong hàm compile, chúng ta sẽ khai báo thêm một chỉ số (metric) mới do Keras hỗ trợ sẵn là SparseTopKCategoricalAccuracy(k=5) để máy tính tự động đo lường Top-5 Accuracy cho chúng ta.

Python

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy',
                       tf.keras.metrics.SparseTopKCategoricalAccuracy(k=5, name='top_5_accuracy')])

print("Bắt đầu huấn luyện...")
# Do mô hình lớn và áp dụng Early Stopping, ta có thể set epochs lớn (ví dụ: 100)
history = model.fit(
    datagen.flow(x_train, y_train, batch_size=128),
    epochs=100,
    validation_data=(x_test, y_test),
    callbacks=[lr_reduction, early_stopping]
)
```

Bước 6: Đánh giá và Phân tích kết quả

Hãy chạy lệnh chấm điểm để xem mô hình của bạn đã vượt qua KPI của đề bài chưa nhé!

Python
```
# Chấm điểm toàn diện trên tập Test
eval_results = model.evaluate(x_test, y_test, verbose=0)
print(f"Độ mất mát (Loss): {eval_results[0]:.4f}")
print(f"Top-1 Accuracy (Đoán trúng phóc): {eval_results[1] * 100:.2f}%")
print(f"Top-5 Accuracy (Đoán trúng trong 5 lựa chọn): {eval_results[2] * 100:.2f}%")

# Vẽ biểu đồ Accuracy
plt.figure(figsize=(12, 5))

# Biểu đồ Top-1 Accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Top-1 Acc')
plt.plot(history.history['val_accuracy'], label='Val Top-1 Acc')
plt.title('Lịch sử Top-1 Accuracy')
plt.legend()
plt.grid(True)

# Biểu đồ Top-5 Accuracy
plt.subplot(1, 2, 2)
plt.plot(history.history['top_5_accuracy'], label='Train Top-5 Acc', color='green')
plt.plot(history.history['val_top_5_accuracy'], label='Val Top-5 Acc', color='red')
plt.title('Lịch sử Top-5 Accuracy')
plt.legend()
plt.grid(True)

plt.show()
```

III. THỬ THÁCH NÂNG CAO

Nếu mô hình tự xây (train from scratch) của bạn đã đạt đến giới hạn (thường là khoảng 55-60% Top-1 Accuracy), làm sao để ứng dụng thực tế có thể đạt **> 80% Top-1 Accuracy**?

Gợi ý Giải pháp: Học chuyển giao (Transfer Learning)

Thay vì huấn luyện từ đầu, hãy sử dụng các siêu mô hình đã được các kỹ sư Google/Microsoft huấn luyện sẵn trên hàng triệu bức ảnh.

- **Yêu cầu:** Hãy tự nghiên cứu tài liệu của Keras (`tf.keras.applications`) để tải một mô hình Pre-trained như **MobileNetV2** hoặc **ResNet50** (không lấy phần đầu - `include_top=False`).
- **Thực hiện:** Đóng băng bộ trọng số của mạng đó lại, thêm một lớp `Dense(100)` vào đuôi và tiến hành huấn luyện để xem kết quả đột phá thế nào nhé!